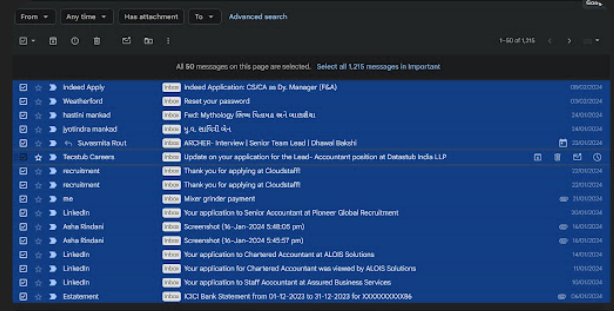




See this screenshot.

Even though I selected all messages you can see that in screenshot that all more than 1000 are selected, when I click three vertical buttons and select as mark not important - the action is taken to mark them only in batch of 100 - I have repeat the process multiple times.

What is the way out? app script?

Gemini 3.6 Flash

In Gmail, bulk toolbar actions like "Mark not important" or "Mark as read" operate only on the threads loaded per page batch, even if you click "Select all 1,215 messages in Important"[cite: 1].

The most effective, automated way to process all matching emails in bulk without manually clicking page by page is to use **Google Apps Script**.

Here is how to set up an automated script to unmark all important emails:

Navigate to script.google.com in your browser and click on ****New project****. Delete any code in the editor (`Code.gs`) and paste the following snippet:

javascript

```
function markAllAsNotImportant() {
  const query = 'is:important'; // Query matches all important emails
  let threads = GmailApp.search(query, 0, 100);

  while (threads.length > 0) {
    GmailApp.markThreadsNotImportant(threads);
    // Fetch the next batch of threads
  }
}
```

```

    threads = GmailApp.search(query, 0, 100);
  }
}

```

Click the ****Save**** icon (disk symbol) or press `Ctrl+S` / `Cmd+S`. Then click the ****Run**** button at the top toolbar. When prompted, click ****Review permissions****, choose your Google account, click ****Advanced****, and click ****Go to Untitled project (unsafe)**** to allow the script to manage your email importance markers.

To verify success, reload your Gmail tab and check the `is:important` view or search[cite: 1]. The message count should be reduced to 0.

Alternative (No-Code Method): Gmail Filters

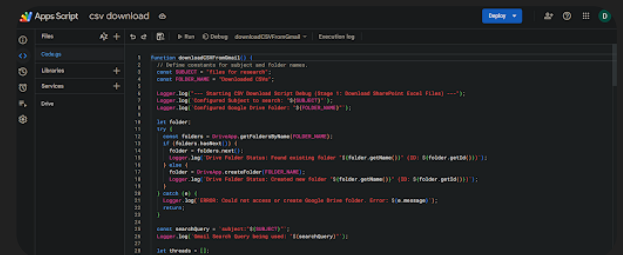
If you prefer not to use Apps Script:

1. Go to Gmail settings (gear icon) > **See all settings** > **Filters and Blocked Addresses**.
2. Create a new filter searching for `is:important` [cite: 1].
3. Click **Continue**, check **Never mark it as important**, check **Also apply filter to matching conversations**, and click **Create filter**.

Would you like help modifying the Apps Script query to target specific senders or date ranges instead of all important emails?

2026-09-23 17:02:19

You 



By miatke I clicked on one of the existing script. Jow to come out. Also tell me step by step workflow for above script. Should I click on new project or what?

 Gemini 3.6 Flash

To exit this existing project, click the **Apps Script icon** (the blue icon with `<>` inside) located at the top-left corner right next to "csv download"[cite: 2]. This will take you back to your main Apps Script dashboard.

Yes, you should click **New project** to keep this script separate from your existing `csv download` project[cite: 2].

Here is the step-by-step workflow:

Click the blue **Apps Script icon** in the top-left corner to return to the project dashboard[cite: 2]. On the main Google Apps Script homepage, click the **+ New project** button on the left sidebar. In the new editor window that opens: 1. Highlight everything in the main editor area (`function myFunction() { ... }`). 2. Delete it and paste the code below:

```
javascript

function markAllAsNotImportant() {
  const query = 'is:important';
  let threads = GmailApp.search(query, 0, 100);

  while (threads.length > 0) {
    GmailApp.markThreadsNotImportant(threads);
    threads = GmailApp.search(query, 0, 100);
  }
}
```

1. Click the **Save** icon (floppy disk) or press `Ctrl + S`. 2. Click the **Run** button on the top toolbar[cite: 2]. 1. Click **Review permissions** when the prompt appears. 2. Select your Google account. 3. Click **Advanced** > **Go to Untitled project (unsafe)**. 4. Click **Allow**.

To verify execution, wait for the script to finish running in the console, then refresh Gmail and check the `is:important` view[cite: 1]. The matching count should be down to 0[cite: 1].

2026-09-23 17:03:23 You 

On clicking `<>` it stays there in editor mode

 Gemini 3.6 Flash

If clicking the `<>` icon keeps you on the same page, it's because that icon in the left sidebar switches the view to the **Editor tab** within the current project[cite: 2].

To return to the main dashboard, click the **Apps Script logo / text** at the top-left corner (above the sidebar) or simply click this link to open the dashboard directly in a new tab: script.google.com.

Alternatively, you don't even need to leave the page. You can paste and run the new script directly inside this same window without creating a new project:

Click the **+** icon next to **Files** in the left sidebar[cite: 2] and select **Script**. Name it `UnmarkImportant`. Paste this code into the editor:

```
javascript

function markAllAsNotImportant() {
  const query = 'is:important';
  let threads = GmailApp.search(query, 0, 100);

  while (threads.length > 0) {
    GmailApp.markThreadsNotImportant(threads);
    threads = GmailApp.search(query, 0, 100);
  }
}
```

1. Click **Save** (Ctrl + S). 2. In the top dropdown menu (which currently shows `downloadCSVFromGmail`), select `markAllAsNotImportant`[cite: 2]. 3. Click **Run**[cite: 2]. To verify execution, wait until the execution log finishes, then refresh Gmail to confirm all important markers are cleared[cite: 1].

2026-09-23 17:04:26

You 

+ icon is new project - But do we need a project?

Gemini 3.6 Flash

In Google Apps Script, a **"Project"** is simply the **file container** that holds your script code. You cannot write or execute code without it being inside a project.

However, you **do not need to create a new project** from scratch. You can simply add a new script file inside your existing project (`csv download`):

Click the **+** icon next to **Files** in the left panel[cite: 2], choose **Script**, and name it `UnmarkImportant`. Paste this code into the new file:

```
javascript

function markAllAsNotImportant() {
  const query = 'is:important';
  let threads = GmailApp.search(query, 0, 100);

  while (threads.length > 0) {
```

```
GmailApp.markThreadsNotImportant(threads);  
threads = GmailApp.search(query, 0, 100);  
}  
}
```

1. Click **Save** (Ctrl + S). 2. Click the function dropdown at the top toolbar (currently showing `downloadCSVFromGmail`) and select `markAllAsNotImportant`. 3. Click **Run**. To verify, wait for the **Execution log** at the bottom to complete, then check your Gmail tab.

2026-09-23 17:06:46

You



Okay saved but I believe, we need to give some permissions to run the script. - Some libraries, etc. I do not exactly remember these nitties.

Gemini 3.6 Flash

You are spot on—you don't need to manually add any external libraries, but Google will ask for **one-time OAuth permissions** to let the script manage your Gmail markers.

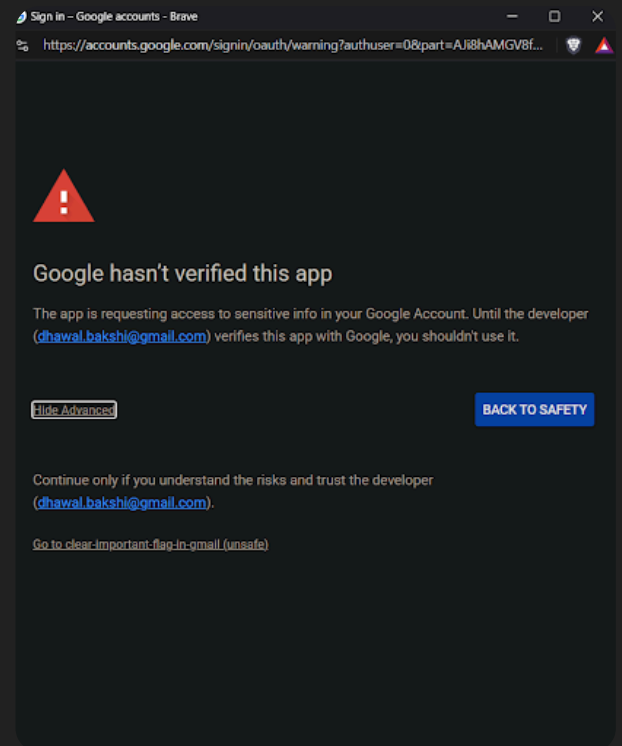
Here is the exact authorization flow to grant permissions:

Click the **Run** button on the top toolbar. A pop-up titled **Authorization required** will appear. Click **Review permissions** in the pop-up box, then select your primary Google account. Google shows a standard safety warning because this is a custom script you wrote yourself. Click **Advanced** at the bottom left of the modal, then click **Go to Untitled project (unsafe)** (or your project name). Scroll to the bottom and click **Allow**. Once permissions are granted, the script won't run automatically—you will need to click **Run** one more time to start the process. To verify it worked, watch the **Execution log** panel at the bottom. Once it displays `Execution completed`, reload Gmail—the `is:important` view will now be empty.

2026-09-23 17:09:08

You





Stuck here. If I click on my account, it leads to dialogue like permission for outlook.com, etc

✦ Gemini 3.6 Flash

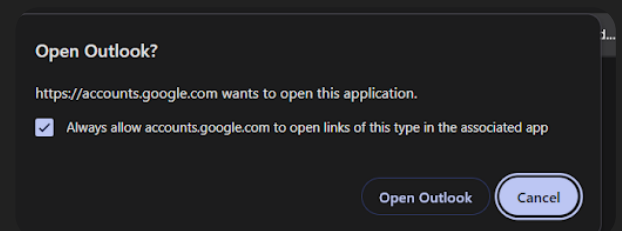
You are right on the final step! This screen is normal for self-created scripts[cite: 3].

Click the link at the bottom left that says **Go to clear-important-flag-in-gmail (unsafe)**[cite: 3].

Click **Go to clear-important-flag-in-gmail (unsafe)**[cite: 3]. On the next screen, scroll down and click **Allow** (or **Continue**) to grant the script access to modify Gmail message tags. Return to the Apps Script tab and click **Run** again if it did not start automatically[cite: 2].

To verify execution, check the **Execution log** at the bottom of Apps Script[cite: 2]. Once it says "Execution completed", refresh Gmail to confirm all items in `is:important` are cleared[cite: 1].

2026-09-23 17:10:16 You 



Again stuck. It asks me to open outlook?

✦ Gemini 3.6 Flash

This popup happens because the link in the previous authorization dialog was a `mailto:` link pointing to your email address (`dhawal.bakshi@gmail.com`), which triggered Windows/Brave to open your default email program (Outlook)[cite: 3, 4].

Here is how to get around it and complete the authorization:

Click **Cancel** on the "Open Outlook?" popup[cite: 4]. Look closely at the very bottom line of text on the previous window[cite: 3].

Do **NOT** click ``dhawal.bakshi@gmail.com``[cite: 3].

Instead, click the **underline link directly below it** that says:
`<u>Go to clear-important-flag-in-gmail (unsafe)</u>`[cite: 3].

1. On the next screen, scroll down and click **Allow** / **Continue**. 2. Switch back to your Google Apps Script tab and click **Run**[cite: 2].

Once the **Execution log** displays `Execution completed` [cite: 2], refresh your Gmail tab[cite: 1]. All messages under `is:important` will be cleared[cite: 1].

2026-09-23 17:10:55

You 

Execution log

17:42:19WarningThis project requires access to your Google Account to run. Please try again and allow it this time.

✦ Gemini 3.6 Flash

That warning means the authorization window was closed before clicking **Allow**.

Because permissions were not completed, Google blocked the script execution.

Here is how to complete the authorization now:

Click the **Run** button at the top toolbar again[cite: 2]. The **Authorization required** window will pop up. Click **Review permissions**, then click on your Google account (`dhawal.bakshi@gmail.com`)[cite: 3]. 1. Click **Advanced** (or **Show Advanced**)[cite: 3]. 2. Click

the link at the very bottom: **Go to clear-important-flag-in-gmail (unsafe)**[cite: 3]. *(Be sure NOT to click your email address link above it, as that opens Outlook[cite: 3, 4]).* 1. Scroll down and click **Allow** (or **Continue**). 2. The window will close, and the script will start automatically. To verify it is running, watch the **Execution log** at the bottom[cite: 2]. You will see it working, and once it prints **Execution completed** [cite: 2], refresh Gmail to confirm all importance tags are gone[cite: 1].

2026-09-23 17:14:18

You 

You allowed clear-important-flag-in-gmail access to some of your Google Account data

 dhawal.bakshi@gmail.com

If you didn't allow clear-important-flag-in-gmail access to some of your Google Account data, someone else may be trying to access your Google Account data.

Take a moment now to check your account activity and secure your account.

[Check activity](#)

To make changes at any time to the access that clear-important-flag-in-gmail has to your data, go to your [Google Account](#)

I received this type of mail but that is okay - I think. IT got executed on selecting unsafe option

 Gemini 3.6 Flash

Yes, that email is completely expected and normal[cite: 5].

Google automatically sends a security notification whenever you authorize a new script or app to access your account data[cite: 5]. Since you wrote and authorized this script yourself, you can safely ignore this alert[cite: 5].

Now that authorization is complete:

1. Switch back to your Google Apps Script tab.
2. Check the **Execution log** at the bottom[cite: 2]. If it says **Execution completed** [cite: 2], the script has finished its task. (If it didn't auto-start, click **Run** one last time[cite: 2]).
3. Refresh your Gmail tab[cite: 1] and perform a search for **is:important** [cite: 1]. All matching emails should now be cleared[cite: 1].



I do not want to remove the script but certainly I wish that none of the messages are marked important by default in future. I do not know how this feature got activated in gmail..

If required you can write another script to remove default marking important

✦ Gemini 3.6 Flash

You don't need a script to stop Gmail from automatically marking future incoming emails as important. This happens because Gmail's built-in "importance prediction" algorithm is turned on by default.

You can turn it off directly in Gmail's settings so future messages are never automatically flagged.

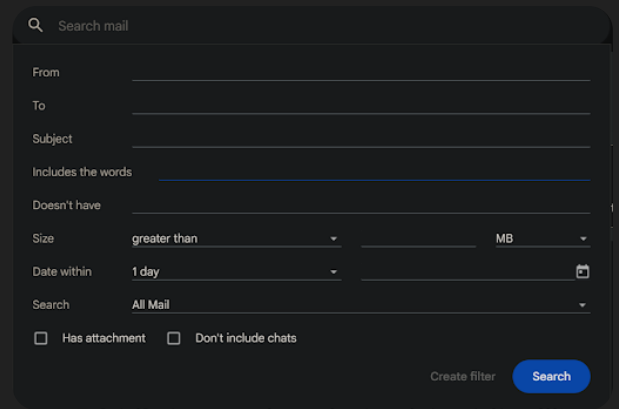
In Gmail, click the **Gear icon** in the top-right corner and select **See all settings**. Click the **Inbox** tab along the top menu bar. 1. Scroll down to the **Importance markers** section. 2. Select **No markers**. 3. Under **Importance predictions**, select **Don't use my past actions to predict which messages are important**. Scroll to the bottom of the page and click **Save Changes**.

Optional: Automatic Override via Filter

If Gmail ever continues to mark certain emails as important due to manual overrides or legacy rules, you can create a single rule to force all incoming mail to be unmarked:

1. Click the **Search options** icon (sliders symbol) inside the Gmail search bar.
2. In the **Has the words** field, type: `size:0` (*this matches 100% of incoming emails*).
3. Click **Create filter**.
4. Check **Never mark it as important**.
5. Click **Create filter**.





you mean this pop up?

✦ Gemini 3.6 Flash

Yes, exactly! That is the correct pop-up[cite: 6].

Here is how to complete the filter setup from where you are right now:

In the **Includes the words** field (where your blue cursor line currently is)[cite: 6], type:

```
`size:0`
```

Click the **Create filter** text button next to the blue Search button at the bottom right[cite: 6]. On the next screen that appears, check the box for **Never mark it as important**. Click the blue **Create filter** button to save the rule.

Once created, Gmail will automatically override its importance algorithms and ensure no incoming emails receive the important marker in the future.

2026-09-23 17:22:09

You 

What extension should I use when saving this script to my local drive .gs?

✦ Gemini 3.6 Flash

When saving your Google Apps Script file locally, you have two options depending on how you plan to use or edit it:

1. **.js (Recommended for local development)**: Standard JavaScript format. If you use code editors like Visual Studio Code, using `.js` gives you full syntax highlighting, formatting, and

linting support. Google Apps Script is based on standard JavaScript.

2. **.gs (For reference / Clasp):** You can save it with the `.gs` extension directly. If you use Google's official command-line tool, **Clasp** (Command Line Apps Script Projects), to sync files between your local drive and Google Drive, Clasp automatically maps local `.js` or `.ts` files to `.gs` on Google's servers.

Bottom line: If you are saving a backup file on your local hard drive or editing in an IDE, save it as **clearImportant.js** (or **clearImportant.gs** if you prefer to visually mark it as a Google Script). Both plain text formats will store your code identically.

2026-09-23 17:23:05

You



I have nodejs with very few npm global modules (inside nodejs folder at D:\Programs\nodejs)

✦ Gemini 3.6 Flash

If you want to sync your Google Apps Script projects locally using Node.js and `npm`, you can use **Clasp** (Google's official **Command Line Apps Script Project** tool).

Here is how you can install and set up Clasp using PowerShell or CMD:

Open PowerShell or Command Prompt and run:

```
powershell  
  
npm install -g @google/clasp
```

This installs the Clasp executable globally so you can access it from any directory.

Run the login command:

```
powershell  
  
clasp login
```

A browser window will open asking for permission to access your Google Apps Script account. Select your account (`dhawal.bakshi@gmail.com`) [cite: 3] and click **Allow**.

Before downloading or uploading scripts, visit [script.google.com/home/usersettings] (<https://script.google.com/home/usersettings>) and toggle **Google Apps Script API** to **ON**. Create a folder on your drive (e.g., `D:\Scripts\clear-important`), open a terminal in that folder, and

run:

```
powershell
```

```
clasp clone "<SCRIPT_ID>"
```

(You can find your `<SCRIPT_ID>` in the Google Apps Script URL:

```
[https://script.google.com/home/projects/](https://script.google.com/home/projects/)<SCRIPT_ID>/edit
```

).

Managing Local Files

- Clasp automatically converts `.gs` files on the web into local `.js` files in your local folder.
- Whenever you make changes locally, run `clasp push` to upload them to Google Apps Script.
- If you edit code in the browser, run `clasp pull` to fetch the latest code to your local drive.

2026-09-23 17:24:41

You



Good to know this. But at present, I just want to save it like any other java script but google app scripts are not exactly javascripts I believe, therefore I created a new folder named "google apps scripts"

✦ Gemini 3.6 Flash

You can save the file as a standard `.js` (or `.gs`) file inside your new `google apps scripts` folder.

While Google Apps Script includes custom global services like `GmailApp` or `DriveApp`, the syntax and runtime are standard **JavaScript (ECMAScript V8)**. Saving it as `.js` gives you full syntax highlighting, auto-formatting, and linting in any local code editor (like VS Code).

Here is how you can organize and save it:

Inside your ``google apps scripts`` folder, create a new file named `**`clearImportant.js`**` (or ``clearImportant.gs``). Paste your standalone script inside:

```
javascript
```

```
/**  
 * Unmarks all important emails in Gmail batch by batch.  
 */
```

```
function markAllAsNotImportant() {
  const query = 'is:important';
  let threads = GmailApp.search(query, 0, 100);

  while (threads.length > 0) {
    GmailApp.markThreadsNotImportant(threads);
    threads = GmailApp.search(query, 0, 100);
  }
}
```

Save the file. Because `.js` and `.gs` are both plain-text files, you can open and edit them anytime with any text editor or IDE.

If you ever want to copy this code back into the browser or push it via command-line tools later, keeping it as a `.js` file ensures your local environment recognizes it cleanly without needing any special plugins.

2026-09-23 17:26:29

You 

Okay did that. Now see this script. What it does:

```
function downloadCSVFromGmail() {
  // Define constants for subject and folder names.
  const SUBJECT = "files for research";
  const FOLDER_NAME = "Downloaded CSVs";

  Logger.log("— Starting CSV Download Script Debug (Stage 1: Download SharePoint Excel Files) —");
  Logger.log(`Configured Subject to search: "${SUBJECT}"`);
  Logger.log(`Configured Google Drive Folder: "${FOLDER_NAME}"`);

  let folder;
  try {
    const folders = DriveApp.getFoldersByName(FOLDER_NAME);
    if (folders.hasNext()) {
      folder = folders.next();
      Logger.log(`Drive Folder Status: Found existing folder "${folder.getName()}" (ID: ${folder.getId()})`);
    } else {
      folder = DriveApp.createFolder(FOLDER_NAME);
      Logger.log(`Drive Folder Status: Created new folder "${folder.getName()}" (ID: ${folder.getId()})`);
    }
  }
}
```

```

    }
  } catch (e) {
    Logger.log(`ERROR: Could not access or create Google Drive
folder. Error: ${e.message}`);
    return;
  }

  const searchQuery = `subject:"${SUBJECT}"`;
  Logger.log(`Gmail Search Query being used: "${searchQuery}"`);

  let threads = [];
  try {
    threads = GmailApp.search(searchQuery);
    Logger.log(`Gmail Search Result: Found ${threads.length} email
threads.`);
  } catch (e) {
    Logger.log(`ERROR: Could not perform Gmail search. Error:
${e.message}`);
    return;
  }

  if (threads.length === 0) {
    Logger.log(`CRITICAL: No emails found matching the subject:
"${SUBJECT}". Double-check subject EXACTNESS.`);
    Logger.log("— Script Finished (No emails found) —");
    return;
  }

  let filesDownloadedCount = 0;

  threads.forEach((thread, threadIndex) => {
    Logger.log(`Processing Thread #${threadIndex + 1} (Subject:
"${thread.getFirstMessageSubject()}", ID: ${thread.getId()})`);
    const messages = thread.getMessages();
    Logger.log(`Found ${messages.length} messages in this
thread.`);

    messages.forEach((message, messageIndex) => {
      Logger.log(`Processing Message #${messageIndex + 1}
(From: ${message.getFrom()}, Date: ${message.getDate()})`);

```

```

    Logger.log(`    Message ID: ${message.getId()}`);
    Logger.log(`    Message is a Draft: ${message.isDraft()}`);
    Logger.log(`    Message is Unread: ${message.isUnread()}`);

    // Attempt with standard getAttachments() first.
    const attachments = message.getAttachments();
    Logger.log(`    Result from message.getAttachments(): Found
    ${attachments.length} attachments.`);

    if (attachments.length === 0) {
        Logger.log(`    No standard attachments found. Attempting
        to extract and download from SharePoint URLs in message
        body...`);
        try {
            const downloadedFromUrls =
            extractAndDownloadFromSharePointUrls(message, folder);
            filesDownloadedCount += downloadedFromUrls;
            if (downloadedFromUrls === 0) {
                Logger.log(`    No files found by extracting and
                downloading from SharePoint URLs either.`);
                Logger.log(`    This confirms the files are likely embedded
                inline (dragged into email body) and/or structured unusually, or
                SharePoint access failed.`);
                Logger.log(`    Final RECOMMENDATION: Please ask the
                sender to attach CSV files using the standard paperclip icon in
                Gmail.`);
            }
        } catch (e) {
            Logger.log(`    ERROR: Could not extract/download from
            URLs. Error: ${e.message}`);
        }
    } else {
        Logger.log(`    Processing standard attachments...`);
        attachments.forEach((file, fileIndex) => {
            const fileName = file.getName();
            const fileType = file.getContentType();
            Logger.log(`    Attachment #${fileIndex + 1}: Name:
            "${fileName}", Type: "${fileType}"`);
            if (fileType === "text/csv" ||
            fileName.toLowerCase().endsWith(".csv")) {

```

```

        try {
            folder.createFile(file);
            Logger.log(`      SUCCESS: Saved "${fileName}" to
Google Drive.`);
            filesDownloadedCount++;
        } catch (e) {
            Logger.log(`      ERROR: Failed to save "${fileName}".
Error: ${e.message}`);
        }
    } else {
        Logger.log(`      SKIPPING: Attachment "${fileName}" is
not a CSV (Type: ${fileType}).`);
    }
});
});
});

if (filesDownloadedCount === 0) {
    Logger.log(`— Script Finished: No files were successfully
downloaded. —`);
} else {
    Logger.log(`— CSV Download Script Complete: Successfully
downloaded ${filesDownloadedCount} files. —`);
}
}

/**
 * Extracts SharePoint URLs from the email's HTML body and
attempts to download them.
 * @param {GoogleAppsScript.Gmail.GmailMessage} message The
Gmail message object.
 * @param {GoogleAppsScript.Drive.Folder} folder The Google Drive
folder to save files to.
 * @returns {number} The number of files successfully extracted
and saved from URLs.
 */
function extractAndDownloadFromSharePointUrls(message, folder)
{
    Logger.log(`    Attempting to extract and download from

```

```

SharePoint URLs...");
let downloadedCount = 0;

// Regex to find <a> tags with href pointing to sharepoint.com and
// capturing the URL and text (filename)
// This regex specifically looks for the filename immediately after
// an <img> tag within the <a>.
const sharepointLinkRegex = /<a[^>]*href="
([^\"]+\.sharepoint\.com[^\"]*)" [^>]*>.*?<img[^\>]*>\s*([^\<]+)?\.\csv)
<\a>/gim;

const htmlBody = message.getBody();
if (htmlBody) {
    Logger.log("    Checking HTML body for SharePoint URLs...");
    let match;
    while ((match = sharepointLinkRegex.exec(htmlBody)) !== null) {
        const url = match[1];
        const fileName = match[2].trim(); // Get the filename from the
link text
        downloadedCount += downloadSharePointFile(url, fileName,
folder);
    }
}

// Also check plain text body for any direct SharePoint URLs,
// though less likely to have names
if (downloadedCount === 0) {
    const plainBody = message.getPlainBody();
    if (plainBody) {
        Logger.log("    Checking plain text body for raw SharePoint
URLs (with filenames)...");
        // This regex captures:
        // 1. Optional icon URL in brackets: (?:\[[^\]]+\])?
        // 2. The filename ending in .csv: ([^\<\s]+)?\.\csv)
        // 3. The SharePoint URL in angle brackets:
<(https?:\:\/\/[^\s>]+sharepoint\.com[^\>]+)>
        const plainTextSharepointLinkRegex = /(?:\[[^\]]+\])?([^\<\s]+)?
\.\csv)<(https?:\:\/\/[^\s>]+sharepoint\.com[^\>]+)>/g;
        let match;
        while ((match = plainTextSharepointLinkRegex.exec(plainBody))

```

```

!= null) {
    const fileName = match[1].trim();
    const url = match[2];
    downloadedCount += downloadSharePointFile(url, fileName,
folder);
    }
    }
    }
    return downloadedCount;
}

/**
 * Attempts to download a file from a SharePoint URL and save it
as an Excel file.
 * This is a staging function to verify initial download success.
 *
 * @param {string} fileUrl The URL of the SharePoint file.
 * @param {string} fileName The desired filename for the
downloaded file (e.g., "2000-2005.csv").
 * @param {GoogleAppsScript.Drive.Folder} folder The Google Drive
folder to save files to.
 * @returns {number} 1 if successfully downloaded, 0 otherwise.
 */
function downloadSharePointFile(fileUrl, fileName, folder) {
    Logger.log(`    Found potential SharePoint file URL: ${fileUrl}`);
    Logger.log(`    Attempting to download and save as temporary
Excel file: "${fileName}"`);

    try {
        const response = UrlFetchApp.fetch(fileUrl, {muteHttpExceptions:
true});

        if (response.getResponseCode() === 200) {
            const excelBlob = response.getBlob();
            // Ensure the temporary file is saved as .xlsx
            const tempExcelFileName = fileName.replace(/\.csv$/i, '.xlsx');
            excelBlob.setName(tempExcelFileName);

            const tempFile = folder.createFile(excelBlob);
            Logger.log(`    SUCCESS: Saved temporary Excel file

```

```

"${tempFile.getName()}" (ID: ${tempFile.getId()}).`);
    return 1; // Success
} else {
    Logger.log("      ERROR: Failed to download "${fileName}"
from ${fileUrl}. Response Code: ${response.getResponseCode()},
Message: ${response.getContentText().substring(0, 200)}...`);
    if (response.getResponseCode() === 401 ||
response.getResponseCode() === 403) {
        Logger.log("      NOTE: A 401/403 error suggests an
authentication/permission issue for the SharePoint file.`);
    }
    return 0;
}
} catch (e) {
    Logger.log("      ERROR: Exception during download:
${e.message}`);
    return 0;
}
// No finally block to trash files, as the goal is to inspect them in
Drive.
}

```

Gemini 3.6 Flash

This script automates searching Gmail for research-related emails, extracting attached or linked CSV/SharePoint files, and saving them directly into a target Google Drive folder.

Core Workflow Breakdown

1. Target Folder Initialization:

- Looks in Google Drive for a folder named **Downloaded CSVs** .
- If it doesn't exist, it automatically creates it to store incoming files.

2. Gmail Email Search:

- Searches Gmail for threads matching the exact subject line: **subject:"files for research"** .

3. Attachment Processing (Primary Path):

- Iterates through every message in the matching email threads.

- Checks for direct email attachments. If it finds files with a `.csv` extension or MIME type `text/csv`, it saves them directly into the Google Drive folder.

4. SharePoint / Link Extraction (Fallback Path):

- If an email contains no standard attachments, it inspects both the **HTML body** and **plain text body** using regular expressions (Regex) to detect embedded SharePoint file links ending in `.csv`.
- When a SharePoint link is found, it attempts to fetch the file via HTTP (`UrlFetchApp.fetch`) and save a copy into Google Drive.

5. Detailed Debug Logging:

- Logs every step—folder creation, thread count, message IDs, attachment types, HTTP response codes, and errors—to the Google Apps Script **Execution Log** for easy troubleshooting.

2026-09-23 17:30:04

You 

Okay. I see like this. Why different symbols for certain scripts

Gemini 3.6 Flash

In the Google Apps Script left sidebar, different symbols next to files indicate the **file type** and **file format**:

- **< >** (**Code / JavaScript**): Represents standard Google Apps Script code files (`.gs`). These contain the functions, business logic, and API calls (like `GmailApp` or `DriveApp`) that perform actions.
- **<html>** or **<> with brackets** (**HTML**): Represents HTML files (`.html`). These are used to render user interfaces, such as custom sidebar panels, pop-up dialog boxes, or web applications hosted on Google Apps Script.
- **JSON / { }** (**Project Manifest**): Represents configuration files like `appscript.json` . This file controls project properties, OAuth scopes, timezone settings, and library dependencies.

These visual distinction icons help you quickly identify logic files (`.gs`), interface templates (`.html`), and project configurations (`.json`) within the same workspace.



Project	Owner
 clear-important-flag-in-gmail	Me
 csv download	Me
 Untitled project	Me
 binary2decimal	Me
 Random bytes to vertical stack	Me
 Solving Equations With 2 Variables	Me

[See like this](#)

✦ Gemini 3.6 Flash

Those icons in your Google Apps Script dashboard distinguish **Standalone Projects** from **Container-Bound Projects**:

- **Blue square with right arrow (→): Standalone Scripts** These exist as independent files directly in Google Drive. They are not tied to any single document, sheet, or form (e.g., `clear-important-flag-in-gmail` and `csv download`)[cite: 7].
- **Green Sheets icon with arrow overlay ( + →): Container-Bound Scripts (Google Sheets)** These scripts were created directly *inside* a Google Sheet (via **Extensions > Apps Script**)[cite: 7]. They are bound to that specific spreadsheet file (like `binary2decimal` or `Solving Equations With 2 Variables`)[cite: 7], allowing them to add custom menu items, custom cell functions, or trigger on edit events in that sheet.



What this one does:

```
function stackData() {
  var sheet = SpreadsheetApp.getActiveSheet();
  var data = sheet.getRange("A1:AD625").getValues();
  var stackedData = [];
  for (var i = 0; i < data.length; i++) {
    for (var j = 0; j < data[i].length; j++) {
      if (data[i][j] !== "") {
        stackedData.push([data[i][j]]);
      }
    }
  }
}
```

```
}  
}  
}  
sheet.getRange(1, 32, stackedData.length,  
1).setValues(stackedData);  
}
```

✦ Gemini 3.6 Flash

This container-bound script reshapes a 2D grid of data from your Google Sheet into a single vertical column.

Core Workflow Breakdown

1. Reads the Data Grid:

- Reads all cell values across the range **A1 to AD625** (30 columns wide by 625 rows down) from the active sheet.

2. Flattens Row-by-Row:

- Iterates through every row, from left to right across columns A through AD.
- Checks each cell and ignores empty cells (`""`).
- Pushes every non-empty value into a new 1D array (`stackedData`).

3. Writes to Column AF:

- Outputs the single consolidated list starting at **Row 1, Column 32** (which corresponds to Column **AF** in Google Sheets).
- Populates as many rows in Column AF as there are non-empty values gathered.

2026-09-23 17:36:56

You 

But is it not 2D and not 3D? I mean columns A to AD data and in multiple rows? What is 3D then?

✦ Gemini 3.6 Flash

Yes, it is **strictly 2D**.

The data exists across **2 axes**:

- 1. **Horizontal axis (X)**: Columns A to AD (30 columns)
- 2. **Vertical axis (Y)**: Rows 1 to 625

When data spans rows and columns on a single worksheet, it forms a **two-dimensional grid (2D matrix)**.

What would 3D Data look like in Spreadsheets?

To make data **3-Dimensional (3D)**, you need a **3rd axis (Z)**, which in spreadsheets represents **depth** (the Sheet tabs).

- **1D (Line)**: Data in a single row or single column (e.g., `A1:A625`).
- **2D (Flat Table)**: Data across multiple rows and columns on **one** sheet (e.g., `A1:AD625` on `Sheet1`).
- **3D (Cube / Stack of Tables)**: The exact same 2D range (`A1:AD625`) evaluated across **multiple sheet tabs** simultaneously (e.g., `Sheet1` through `Sheet12`).

2D Grid (Single Sheet)					3D Cube (Multiple Sheets)				
A	B	C	...	AD	Jan	Feb	Mar		
1	[]	[]	[]	[]	1 []	[]	[]		
2	[]	[]	[]	[]	2 A1	A1	A1		
3	[]	[]	[]	[]	3 []	[]	[]		
...									
625	[]	[]	[]	[]					

Example of a 3D Formula

If you have monthly sales tabs named `Jan` , `Feb` , `Mar` ... through `Dec` , and you want to sum cell `A1` across all 12 months, you write a **3D Reference**:

```
excel
=SUM( Jan:Dec!A1)
```

Here, `A1` is the 2D coordinate on each page, and `Jan:Dec` is the 3rd dimension slicing through the stack of sheets.

In your script, because it only reads from one active worksheet, it is taking a **2D matrix** (`data[row][column]`) and converting it into a **1D array** (a single column).

For example I have A to AD and then AG to BB and multiple rows in same sheet then also is it not 3D?

✦ Gemini 3.6 Flash

No, that is still **2D**.

Having multiple separate blocks of columns on the same sheet (like `A:AD` and `AG:BB`) just means you have **non-contiguous (disjoint) 2D ranges** on the same flat surface.

Think of a piece of graph paper:

- **1D:** Drawing a single straight line along one grid line (length only).
- **2D:** Drawing a shape or filling cells anywhere on that same flat sheet of paper (width and height). It doesn't matter if you draw one big box, two boxes with a gap in between, or a hundred scattered boxes—as long as it's on the same sheet of paper, it remains two-dimensional because any cell's position is completely defined by just **two coordinates: (Row, Column)**.

To make it **3D**, you have to leave the page and add **depth**:

2D (One Sheet)	3D (Multiple Sheets)
Row & Column only Coordinate: (Row 10, Col B)	Row, Column, AND Page/Tab Coordinate: (Sheet "Jan", Row 10, Col B)
+-----+ A..AD AG..BB [] [] [] [] +-----+	+-----+ <- Sheet "Mar" / A..AD AG..BB / <- Sheet "Feb" +-----+ <- Sheet "Jan" A..AD AG..BB [] [] [] [] +-----+

Summary Comparison

Concept	Spreadsheet Equivalent	Coordinates Needed
1D	A single row or a single column (<code>A1:A100</code>)	1 (<code>Row</code>) or (<code>Column</code>)
2D	Any range, multiple blocks, or entire grid on one sheet (<code>A1:AD625</code> , <code>AG1:BB625</code>)	2 (<code>Row</code> , <code>Column</code>)

Conc ept	Spreadsheet Equivalent	Coordinates Needed
3D	Any range evaluated across multiple sheet tabs at once (Sheet1:Sheet12!A1:AD625)	3 (Sheet , Row , Column)